

Johnson-Lindenstrauss Lemma

Following all the given steps, we obtain two sets of data: the initial data-set X and the compressed data-set Y , which was created as a linear transformation of X ($Y = WX$, where W is a random projection matrix).

The initial data-set has data-points (*vectors*) of high dimension d , while the ones of the compressed data set, contain less information, since they exist in the n -dimensional space $\mathbb{R}^n$, for which holds $n < d$. Note that since n is derived from a formula that d is not contained ($n = \frac{6 \log(\frac{2Q}{\delta})}{\epsilon^2}$), the choice of ϵ , δ is important so that we get n such that $n < d$.

For both data-sets, we compute the pairwise differences of all the $\frac{m(m-1)}{2}$ possible combinations of vectors contained in each data set, where m is the number of data-vectors. This way, when plotting the compressed data in a common plot with the original data multiplied by a factor of $\sqrt{1 \pm \epsilon}$ we can visually check and confirm the correctness of our implementation, with respect to what the *Johnson-Lindenstrauss Lemma* states.

The essence of the JJ Lemma lies on the fact that the projections of a finite set of d -dimensional vectors using a random projection W , will not deviate further from $\sqrt{1 \pm \epsilon}$ from the original vector, with probability at least $1 - \delta$, which in math translates to:

$$Pr \left[\sup_{x \in \mathbb{Q}} \left| \frac{\|Wx\|^2}{\|x\|^2} - 1 \right| < \epsilon \right] \geq 1 - \delta$$

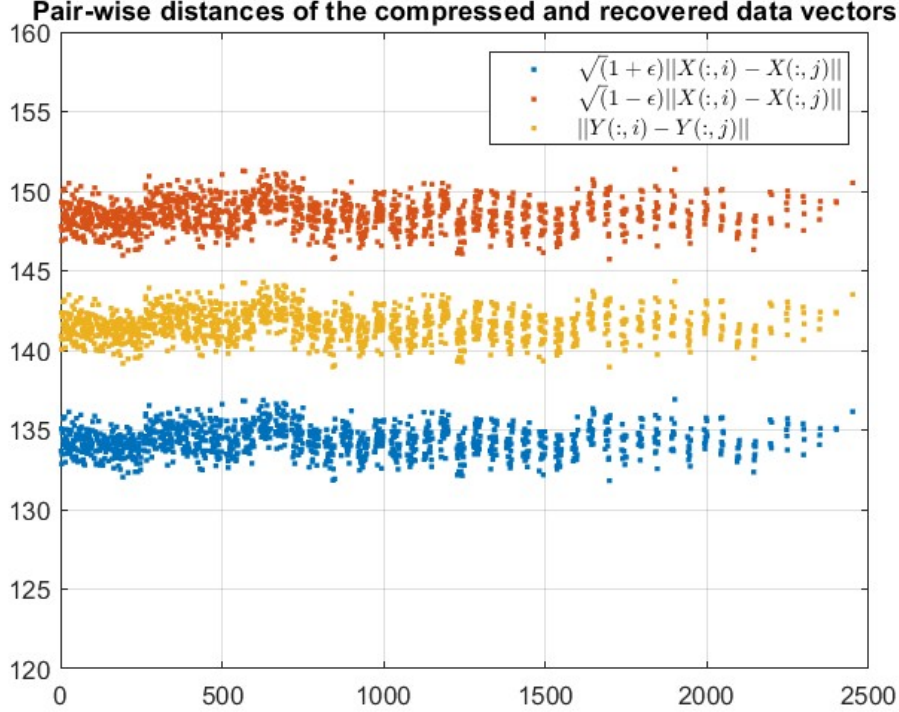


Figure 1: Plot of the difference between the original data vectors of X , scaled by $\sqrt{1 \pm \epsilon}$, and the difference between the compressed data vectors of Y

Indeed after checking the plot, we can say that the event of finding any difference of the projected set to exceed the $\sqrt{1 \pm \epsilon}$ scaled differences of the non-projected set is highly improbable.

Also, after computing and printing the empirical probability of success, in the MATLAB command window, we can confirm the above statement.

Empirical Probability of Success = 1.00

The calculation was made by counting how many of the projected differences fall inside the bounds of the original and scaled by $\sqrt{1 \pm \epsilon}$ differences and then dividing by the total number of possible differences.

In other words, we counted the occurrences that the projected differences exceed the limits we set and found what we mathematically described earlier, that is, such an occurrence is very rare.

2-SAT Randomized Algorithm

We start by setting up the formula consisting of the conjunctions of all the clauses. Each of the clauses includes a combination of two boolean variables, that is never repeated throughout the formula, in any form (negated or not). Using an arbitrary truth assignment s , we can determine the form in which the literals in each clause are included. After that, we have the complete formula and the only thing left is to apply the *2-SAT Randomized Algorithm* on another arbitrary truth assignment x , so that it is after the execution of the algorithm, x is modified into a satisfying truth assignment for the boolean formula.

After running the algorithm for different combinations of n, K and m , we can see that the algorithm almost never fails to find a satisfying truth assignment (starting from an arbitrary one and modifying it in each iteration). As the Theorem in our theory states, our belief that the randomized algorithm actually found a satisfying truth assignment is pretty strong, since the probability of such a mistake is at most $\frac{1}{2^m}$. This means that even at the lowest possible reliability $m = 1$ (being an integer), we can be adequately sure that the algorithm will mostly be truthful when returning an assignment as satisfying.

Moreover, we can see that every time the algorithm is ran, the number of iterations needed, for a successful execution, are bounded well below from n^2 .

Finally, for a certain number of clauses K , although the algorithm may not be able to find a solution for a given reliability m , by increasing m , we may be able to find a solution, and that's because the algorithm has more time ($2mn^2$ iterations) to modify the arbitrary truth assignment we started with and end up in a truthful assignment, if one can be found.